

Freight Rate Prediction

From 48,000 historical loads to 12,000 future-rate predictions

95.50

final average walk-forward MAE (\$)

12,000

validation loads predicted

48,000

historical labeled loads

101.02

December-compatible average MAE (\$)

Bottom line: The final model predicts rate per mile with XGBoost, then multiplies by distance to return the shipment rate. The validation design is time-based so every fold predicts a month that comes after its training history.

What this report focuses on

The goal here is not to list every experiment I ran. It is to show the decisions that materially changed the model: how I split the data, why the target became rate per mile, which features survived validation, and how the final prediction workflow was chosen.

A few data checks changed the modeling choices

The training file contains 48,000 labeled loads from January through October 2025. Before fitting models, I checked missing values, outliers, categorical coverage, and physically invalid values, then applied the necessary cleaning before modeling.

Check	What I found	What I did
Missing weight	300 training rows	Median imputation inside the pipeline
Missing market_index	374 training rows	Tested it, but it was excluded from the final feature set
Invalid negative weight	292 training rows	Corrected before modeling using absolute values
Unseen locations	8 city names appear in validation but not training	OneHotEncoder(handle_unknown="ignore")
High rate/mile	317 rows above \$5/mile	Kept them; filtering did not materially improve validation

I kept the calendar intact

A random split would let January and October loads mix together, which is not how the model will be used. The hidden set represents future loads, so I used an expanding-window validation: train only on the past, then score the next month.

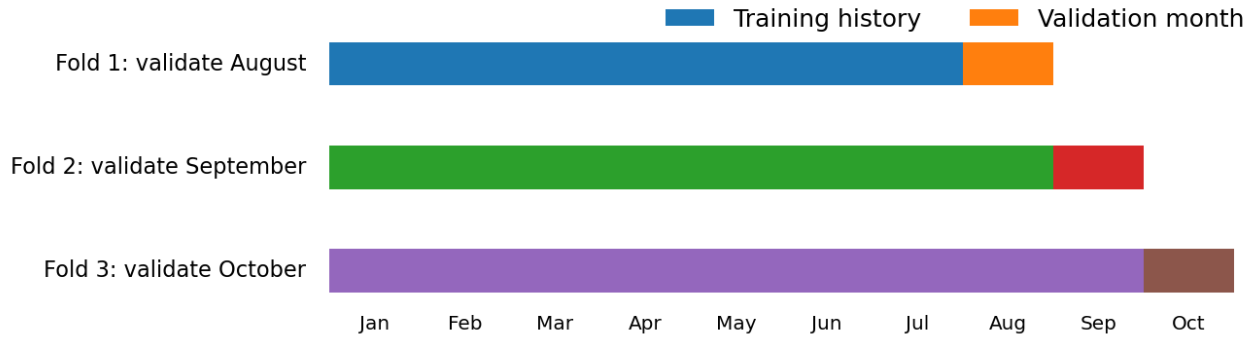


Figure 1. Expanding-window validation used for model selection and tuning.

Fold	Training data	Validation data	Why it matters
1	Jan 1 - Jul 31	August	First forward-looking check
2	Jan 1 - Aug 31	September	Adds one more month of history
3	Jan 1 - Sep 30	October	Closest proxy to future deployment

I used mean absolute error (MAE) as the main selection metric because the output is a dollar rate. A MAE of \$95.50 means the prediction misses the actual rate by about \$95.50 on average across these validation folds, without giving extra weight to a few large misses.

The biggest modeling gain came from changing the target

Distance is the dominant driver of total freight rate. Instead of asking the model to learn the full dollar amount directly, I also tested predicting dollars per mile and then converting the result back to a total rate.

Prediction path: $\text{rate_per_mile} = \text{posted_rate} / \text{distance}$ -> XGBoost predicts rate_per_mile -> $\text{predicted_rate} = \text{predicted_rate_per_mile} \times \text{distance}$

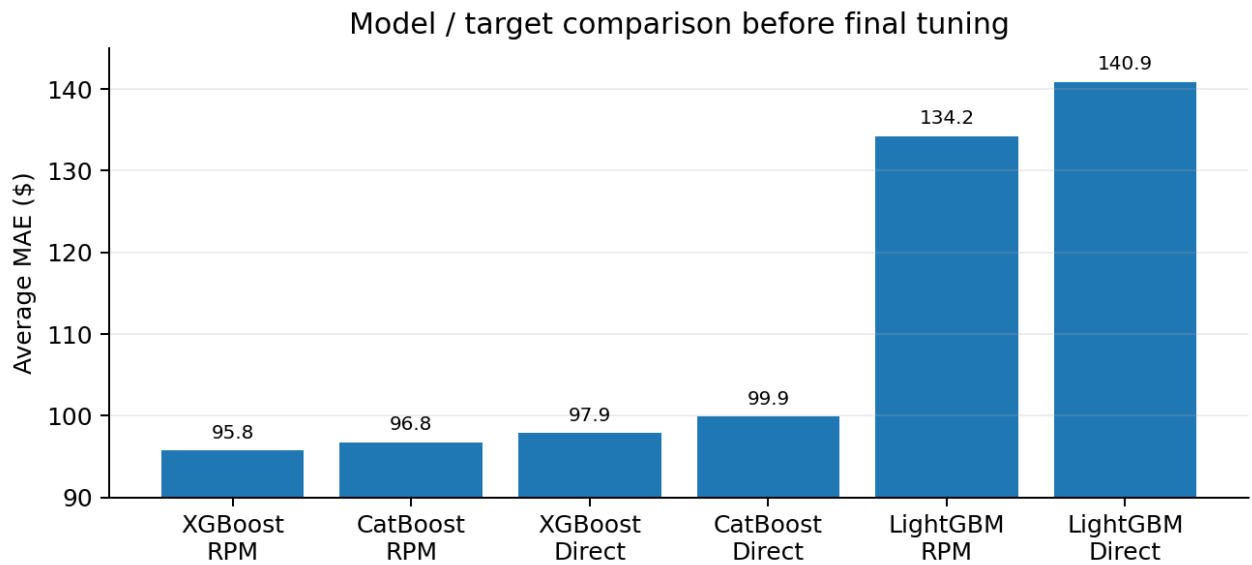


Figure 2. Average MAE from the model/target comparison stage. XGBoost + rate per mile was selected for final tuning.

Configuration	Average MAE (\$)	Decision
XGBoost + Rate Per Mile	95.76	Selected for tuning
CatBoost + Rate Per Mile	96.76	Strong alternative
XGBoost + Direct Rate	97.93	Worse than RPM target
CatBoost + Direct Rate	99.89	Worse than RPM target
LightGBM + Rate Per Mile	134.23	Not competitive
LightGBM + Direct Rate	140.86	Not competitive

Note: these are the configurations tested during the comparison stage, not a claim that one model family is universally better. XGBoost was the strongest configuration in this experiment, so it received the later tuning work.

The final feature set is smaller than the starting set

I used ablation tests rather than correlation alone. That matters because tree models can use interactions and nonlinear splits that a single correlation coefficient will miss.

Feature group	Final choice	Reason
Locations	pickup, delivery, route	Kept route context and both endpoint identities
Coordinates	pickup/delivery lat + lon	Improved the rate-per-mile validation result
Shipment	distance, equipment, weight	Core pricing signals
Calendar	day_of_week only	Removing month and day_of_year improved the main model
Market signals	Dropped	market_index and quote_signal hurt RPM validation

Final 11 features

pickup, delivery, route, equipment, distance, weight, pickup_lat, pickup_lon, delivery_lat, delivery_lon, day_of_week

Final XGBoost setting	Value
n_estimators	1000
max_depth	5
learning_rate	0.05
subsample / colsample_bytree	0.90 / 0.90
objective	reg:absoluteerror
tree_method	hist
random_state	42

I stopped tuning at this point because the August-October folds had already been reused for several decisions; more searching would risk tuning to those months instead of improving future performance.

The final setup stayed consistent across all three validation months

Validation month	Final MAE (\$)
August	91.72
September	95.45
October	99.32
Average	95.50

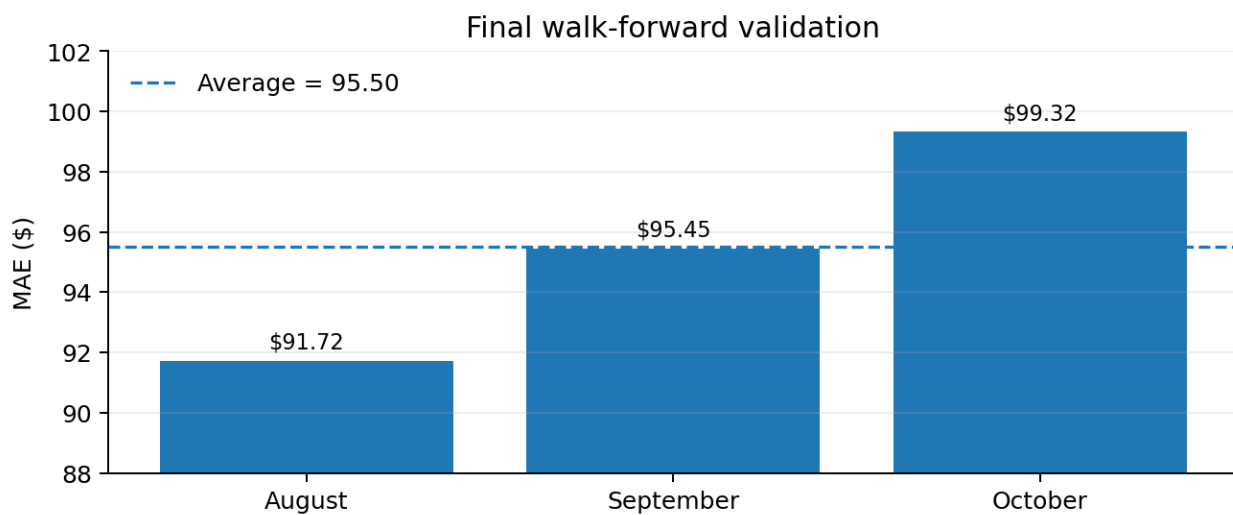


Figure 3. Final XGBoost + rate-per-mile walk-forward MAE.

Final main model: XGBoost + rate-per-mile target + 11 features + 1000 trees. Average walk-forward MAE = \$95.50.

The final model is then retrained on all 48,000 labeled rows and used to generate one rate for each of the 12,000 validation load IDs. The output file keeps only `load_id` and `predicted_rate`, matching the supplied template.

A separate compatible model handles the fixed December route

The December input does not contain the latitude/longitude fields used by the main model, so I trained a second model using only fields available in that file. It keeps the same XGBoost + rate-per-mile idea but uses pickup, delivery, route, equipment, distance, weight, month, day_of_week, and day_of_year.

Month	December-model MAE (\$)
August	86.32
September	102.68
October	114.08
Average	101.02

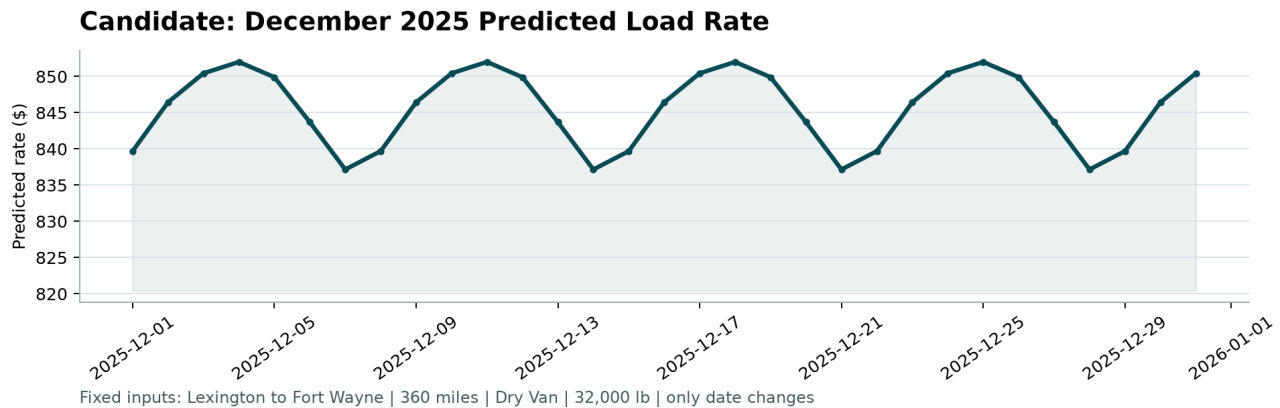


Figure 4. Candidate December 2025 predicted load rate for Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb. Only the date changes.

The repeated weekly shape is expected. Every non-date input is fixed, so day-of-week becomes the main changing signal. Because the estimator is tree-based, I would not interpret this as a smooth time trend beyond the observed feature patterns.

The final run is intentionally short

Once the experiments are done, only three commands are needed to rebuild the deliverables from the project root:

```
python src/08_generate_validation_predictions.py
```

```
python src/09_december_model.py
```

```
python score.py --predictions validation_predictions.csv --december-predictions data/december_chart_inputs.csv
```

Deliverable	Purpose
validation_predictions.csv	12,000 validation load IDs with predicted rates
data/december_chart_inputs.csv	31 December rows with predicted_rate filled
scorer_results/candidate_december.png	Required chart generated by the scorer
src/	Reproducible analysis, validation, tuning, and final prediction scripts

Final takeaway: The best result came from keeping the validation realistic, using rate per mile as the target, and retaining only the features that continued to work on future-month validation.

Final walk-forward MAE: \$95.50 | December-compatible MAE: \$101.02